

Psych Mission Simulator - Installation Guide

Overview

This document provides instructions to set up, install, and run the Psyche Mission Simulator application. Follow the steps below to ensure a smooth setup process.

System Requirements

Before proceeding, verify that your system meets the following requirements:

- Operating System: Windows 10/11, macOS, or Linux
 - Node.js: v16.x or higher (for frontend)
 - npm: v8.x or higher
 - .NET SDK: .NET 6.0 or higher
 - PostgreSQL : v13 or higher
 - Other software: Git, Docker (optional for containerization)
-

1. Clone the Repository

Start by cloning the project from the repository:

Copy code

```
git clone https://github.com/username/project-name.git
cd project-name
```

2. Install Dependencies

Backend Installation

The backend for this project is built using ASP.NET Core. Follow these steps to install it:

1. Install .NET SDK:
 - Download and install the .NET SDK from [Microsoft's .NET Download page](#).
2. Navigate to the backend directory:

Copy code

```
cd backend
```

3. Restore dependencies:

Copy code

```
dotnet restore
```

Frontend Installation

The frontend uses HTML5, CSS3, JavaScript, React, and Vite for a fast development setup. Follow these steps to set up:

1. Ensure Node.js and npm are installed:
 - Download from [Node.js website](#).
 - Verify installation with:

Copy code

```
node -v
```

```
npm -v
```

2. Navigate to the frontend directory:

Copy code

```
cd ../frontend
```

3. Install the necessary dependencies:

Copy code

```
npm install
```

3. Configure Environment Variables

Backend Configuration

1. In the backend directory, create an environment file (appsettings.Development.).
2. Add your database configuration:

Copy code

```
{
  "ConnectionStrings": {
    "DefaultConnection":
      "Host=localhost;Database=[database_name];Username=[username];Password=[password]"
  },
  "JwtSettings": {
    "Secret": "[Your secret key]"
  }
}
```

Frontend Configuration

1. In the frontend directory, create a .env file from the example provided:

Copy code

```
cp .env.example .env
```

2. Update the necessary environment variables:

Copy code

```
VITE_API_URL=http://localhost:[backend_port]/api
```

4. Set Up the Database

The database for this project is Postgre . Ensure Postgre is installed and running.

1. Download and install Postgre :
 - Download from [Postgre website](#).
2. Create a database:
 - Start the Postgre shell and enter:

Copy code

```
CREATE DATABASE [database_name];
```

3. Apply database migrations (if applicable):

Copy code

```
cd backend
dotnet ef database update
```

5. Running the Application

Start the Backend

To start the ASP.NET Core backend server:

Copy code

```
cd backend
dotnet run
```

The backend server will be running on `http://localhost:[port]`.

Start the Frontend

To run the frontend React app:

Copy code

```
cd frontend
npm run dev
```

The frontend app will be available at <http://localhost:3000>.

6. Testing

Open a browser and navigate to the frontend URL to ensure both the frontend and backend are properly connected.

7. Building for Production

Frontend Build

When ready to deploy the frontend, create a production build:

Copy code

```
cd frontend
npm run build
```

Backend Start for Production

To start the backend in production mode:

Copy code

```
cd backend
dotnet publish --configuration Release
```

8. Troubleshooting

Common Issues:

- Backend not connecting to the database: Ensure database credentials are correct in the `appsettings.Development.` .
 - Frontend not loading API data: Verify `VITE_API_URL` is correctly pointing to the backend URL.
-

9. Additional Resources

For further information, visit:

- [React Documentation](#)
- [.NET Documentation](#)
- [Postgre Documentation](#)